

Tactical Vehicle Simulation Backend

Project Overview

This application simulates vehicle movement on predefined routes.

Features:

- Create Plans
 - Assign Vehicles to Plans
 - Start Simulation
 - Stop Simulation
 - Vehicle Movement Scheduler
 - Fuel Consumption Tracking
 - Distance Covered Tracking
 - Distance Remaining Tracking
 - Time Taken Tracking
 - PostgreSQL Persistence
-

Technology Stack

- Java 25
 - Spring Boot
 - Spring Data JPA
 - PostgreSQL
 - Lombok
 - Maven
-

Prerequisites

Java

Verify Java:

```
java --version
```

Expected:

```
Java 25
```

PostgreSQL

Install PostgreSQL.

Verify:

```
psql --version
```

Database Setup

Create Database

```
CREATE DATABASE tactical_db
WITH
OWNER = postgres
ENCODING = 'UTF8';
```

application.properties

```
spring.application.name=demo

server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/tactical_db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.idle-timeout=30000
spring.datasource.hikari.connection-timeout=20000

logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

```
spring.mvc.async.request-timeout=0
```

Build Application

```
mvn clean package
```

Generated Jar:

```
target/demo-0.0.1-SNAPSHOT.jar
```

Run Application

```
java -jar demo-0.0.1-SNAPSHOT.jar
```

Application URL:

```
http://localhost:8080
```

Vehicle Bootstrap

On application startup:

Vehicle	Type
ARMY_VEHICLE	ARMY_VEHICLE
TANK	TANK
TRUCK	TRUCK

These records are automatically inserted by DataInitializer.

API Endpoints

1. Create Plan A

POST

<http://localhost:8080/api/plans>

Request:

```
{
  "planName": "PLAN_A",
  "speed": 50,
  "startTime": "2026-06-10T10:00:00",
  "routePoints": [
    {
      "latitude": 28.6139,
      "longitude": 77.2090
    },
    {
      "latitude": 28.6200,
      "longitude": 77.2150
    },
    {
      "latitude": 28.6300,
      "longitude": 77.2200
    },
    {
      "latitude": 28.6400,
      "longitude": 77.2250
    },
    {
      "latitude": 28.6500,
      "longitude": 77.2300
    },
    {
      "latitude": 28.6600,
      "longitude": 77.2350
    },
    {
      "latitude": 28.6700,
      "longitude": 77.2400
    }
  ]
}
```

2. Create Plan B

POST

<http://localhost:8080/api/plans>

Request:

```
{
  "planName": "PLAN_B",
  "speed": 60,
  "startTime": "2026-06-10T10:05:00",
  "routePoints": [
    {
      "latitude": 28.7041,
      "longitude": 77.1025
    },
    {
      "latitude": 28.7000,
      "longitude": 77.1100
    },
    {
      "latitude": 28.6950,
      "longitude": 77.1200
    },
    {
      "latitude": 28.6900,
      "longitude": 77.1300
    },
    {
      "latitude": 28.6850,
      "longitude": 77.1400
    },
    {
      "latitude": 28.6800,
      "longitude": 77.1500
    },
    {
      "latitude": 28.6750,
      "longitude": 77.1600
    }
  ]
}
```

3. Create Plan C

POST

<http://localhost:8080/api/plans>

Request:

```
{
  "planName": "PLAN_C",
  "speed": 40,
  "startTime": "2026-06-10T10:10:00",
  "routePoints": [
    {
      "latitude": 28.5355,
      "longitude": 77.3910
    },
    {
      "latitude": 28.5400,
      "longitude": 77.3950
    },
    {
      "latitude": 28.5450,
      "longitude": 77.4000
    },
    {
      "latitude": 28.5500,
      "longitude": 77.4050
    },
    {
      "latitude": 28.5550,
      "longitude": 77.4100
    },
    {
      "latitude": 28.5600,
      "longitude": 77.4150
    },
    {
      "latitude": 28.5650,
      "longitude": 77.4200
    }
  ]
}
```

Start Simulation

POST

```
http://localhost:8080/api/simulation/start
```

What happens:

- Plan A assigned to Vehicle 1
 - Plan B assigned to Vehicle 2
 - Plan C assigned to Vehicle 3
 - Scheduler starts
 - Vehicles begin moving
-

Stop Simulation

POST

```
http://localhost:8080/api/simulation/stop
```

What happens:

- Scheduler stops
 - All vehicles marked as stopped
-

Get All Plans

GET

```
http://localhost:8080/api/plans/getAllPlans
```

Sample Response:

```
[
  {
    "planId": 1,
    "planName": "PLAN_A",
    "speed": 50,
    "status": "CREATED",
    "vehicle": {
      "vehicleName": "ARMY_VEHICLE",
```

```
        "fuelRemaining": 999.88,  
        "distanceCoveredKm": 6.96,  
        "distanceRemainingKm": 0.00,  
        "timeTakenSeconds": 120,  
        "simulationStarted": false  
    }  
}  
]
```

Simulation Logic

Scheduler executes every:

```
@Scheduled(fixedRate = 1000)
```

Meaning:

- Every 1 second
- Vehicle moves to next route point
- Fuel decreases
- Distance covered increases
- Remaining distance recalculated

Fuel Consumption

Current implementation:

```
1 Liter / Minute
```

Per scheduler tick:

```
1.0 / 60.0
```

liters consumed.

Distance Calculation

Distance calculated using:

Haversine Formula

Used for:

- Distance Covered
- Distance Remaining

Deployment Checklist

✓ Java 25 Installed

✓ PostgreSQL Installed

✓ tactical_db Created

✓ Tables Auto Created

✓ application.properties Updated

✓ Build Successful

✓ demo-0.0.1-SNAPSHOT.jar Generated

✓ Run:

```
java -jar demo-0.0.1-SNAPSHOT.jar
```

✓ Create Plans

✓ Start Simulation

✓ Verify Movement

✓ Stop Simulation

✓ Fetch Plan Details

Project Status: Ready for POC Demonstration